

## ■ LEVEL 20 • 2D DP

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

|                           |                                                                                                        |
|---------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Pattern Trigger</b>    | Two changing variables describe the state, often represented by a table over two positions/parameters. |
| <b>Reference Examples</b> | LCS • Edit Distance • Unique Paths • Minimum Path Sum • 0/1 Knapsack                                   |
| <b>Problems</b>           | 6 focused problems in this level                                                                       |
| <b>Code Standard</b>      | C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings                  |

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

## LEVEL 20 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

| #  | Problem                         | Pattern | LeetCode |
|----|---------------------------------|---------|----------|
| 1  | Unique Paths                    | 2D DP   | #62      |
| 2  | Minimum Path Sum                | 2D DP   | #64      |
| 3  | Longest Common Subsequence      | 2D DP   | #1143    |
| 4  | Edit Distance                   | 2D DP   | #72      |
| 5  | Interleaving String             | 2D DP   | #97      |
| 6  | Distinct Subsequences           | 2D DP   | #115     |
| 7  | Unique Paths                    | Grid DP | 62       |
| 8  | Minimum Path Sum                | Grid DP | 64       |
| 9  | Longest Common Subsequence      | 2D DP   | 1143     |
| 10 | Longest Palindromic Subsequence | 2D DP   | 516      |

**Extension rule:** Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

# PROBLEM 1 Unique Paths

Pattern: 2D DP

LeetCode: #62

|                       |                                                                        |
|-----------------------|------------------------------------------------------------------------|
| ■ Input               | m x n grid                                                             |
| ■ Output              | number of paths from top-left to bottom-right                          |
| ■ Constraints         | m,n ≤ 100                                                              |
| ■ Main Idea / Pattern | Each cell receives paths from the cell above and the cell to the left. |
| ■■ Time Complexity    | O(mn)                                                                  |
| ■ Space Complexity    | O(n)                                                                   |

## ■ Optimal C++ Code

```
int uniquePaths(int m, int n) {  
    vector<int> dp(n, 1);  
  
    for (int r = 1; r < m; r++)  
        for (int c = 1; c < n; c++)  
            dp[c] += dp[c - 1];  
  
    return dp[n - 1];  
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 2 Minimum Path Sum

Pattern: 2D DP

LeetCode: #64

|                       |                                                           |
|-----------------------|-----------------------------------------------------------|
| ■ Input               | non-negative grid                                         |
| ■ Output              | minimum path sum to bottom-right                          |
| ■ Constraints         | $m, n \leq 200$                                           |
| ■ Main Idea / Pattern | At each cell, choose the cheaper path from above or left. |
| ■ Time Complexity     | $O(mn)$                                                   |
| ■ Space Complexity    | $O(n)$                                                    |

### ■ Optimal C++ Code

```
int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    vector<int> dp(n, INT_MAX);

    dp[0] = 0;

    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            if (c > 0)
                dp[c] = min(dp[c], dp[c - 1]);

            dp[c] += grid[r][c];
        }
    }

    return dp[n - 1];
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 3 Longest Common Subsequence

Pattern: 2D DP

LeetCode: #1143

|                       |                                                                     |
|-----------------------|---------------------------------------------------------------------|
| ■ Input               | two strings                                                         |
| ■ Output              | LCS length                                                          |
| ■ Constraints         | lengths $\leq 1000$                                                 |
| ■ Main Idea / Pattern | $dp[i][j]$ is the best answer for prefixes of lengths $i$ and $j$ . |
| ■ Time Complexity     | $O(nm)$                                                             |
| ■ Space Complexity    | $O(nm)$                                                             |

### ■ Optimal C++ Code

```
int longestCommonSubsequence(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }

    return dp[n][m];
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 4 Edit Distance

Pattern: 2D DP

LeetCode: #72

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| ■ Input               | two strings                                                                             |
| ■ Output              | minimum insertions, deletions, replacements                                             |
| ■ Constraints         | lengths $\leq 500$                                                                      |
| ■ Main Idea / Pattern | If characters match, use diagonal. Otherwise take 1 + minimum of three edit operations. |
| ■ Time Complexity     | $O(nm)$                                                                                 |
| ■ Space Complexity    | $O(nm)$                                                                                 |

### ■ Optimal C++ Code

```
int minDistance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));

    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1];
            else
                dp[i][j] = 1 + min({
                    dp[i - 1][j],
                    dp[i][j - 1],
                    dp[i - 1][j - 1]
                });
        }

    return dp[n][m];
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 5 Interleaving String

Pattern: 2D DP

LeetCode: #97

|                       |                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------|
| ■ Input               | three strings s1,s2,s3                                                                |
| ■ Output              | whether s3 is an interleaving of s1 and s2                                            |
| ■ Constraints         | length sum must match                                                                 |
| ■ Main Idea / Pattern | dp[i][j] means first i chars of s1 and first j chars of s2 can form the prefix of s3. |
| ■ Time Complexity     | O(nm)                                                                                 |
| ■ Space Complexity    | O(nm)                                                                                 |

### ■ Optimal C++ Code

```
bool isInterleave(string a, string b, string c) {
    if (a.size() + b.size() != c.size())
        return false;

    vector<vector<bool>> dp(a.size() + 1,
                          vector<bool>(b.size() + 1));

    dp[0][0] = true;

    for (int i = 0; i <= a.size(); i++)
        for (int j = 0; j <= b.size(); j++) {
            int k = i + j - 1;

            if (i > 0)
                dp[i][j] = dp[i][j] ||
                    (dp[i - 1][j] && a[i - 1] == c[k]);

            if (j > 0)
                dp[i][j] = dp[i][j] ||
                    (dp[i][j - 1] && b[j - 1] == c[k]);
        }

    return dp[a.size()][b.size()];
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 6 Distinct Subsequences

Pattern: 2D DP

LeetCode: #115

|                       |                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------|
| ■ Input               | strings s and t                                                                        |
| ■ Output              | number of distinct subsequences of s equal to t                                        |
| ■ Constraints         | lengths $\leq 1000$                                                                    |
| ■ Main Idea / Pattern | If chars match, use both take and skip possibilities; otherwise skip s's current char. |
| ■■ Time Complexity    | $O(nm)$                                                                                |
| ■ Space Complexity    | $O(nm)$                                                                                |

### ■ Optimal C++ Code

```
long long numDistinct(string s, string t) {  
    int n = s.size(), m = t.size();  
    vector<vector<long long>> dp(n + 1,  
                                vector<long long>(m + 1));  
  
    for (int i = 0; i <= n; i++)  
        dp[i][0] = 1;  
  
    for (int i = 1; i <= n; i++)  
        for (int j = 1; j <= m; j++) {  
            dp[i][j] = dp[i - 1][j];  
  
            if (s[i - 1] == t[j - 1])  
                dp[i][j] += dp[i - 1][j - 1];  
        }  
  
    return dp[n][m];  
}
```

**Practice note:** Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

## PROBLEM 7 Unique Paths

Pattern: Grid DP

LeetCode: 62

|                     |                                                                       |
|---------------------|-----------------------------------------------------------------------|
| Input               | m, n                                                                  |
| Output              | number of paths from top-left to bottom-right                         |
| Constraints         | 1 <= m,n <= 100                                                       |
| Main Idea / Pattern | Each cell is reachable from the top or left, so add those two states. |
| Time Complexity     | O(mn)                                                                 |
| Space Complexity    | O(n)                                                                  |

### Optimal C++ Code

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[j] += dp[j - 1];
            }
        }
        return dp[n - 1];
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 8 Minimum Path Sum

Pattern: Grid DP

LeetCode: 64

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| Input               | grid                                                      |
| Output              | minimum sum path from top-left to bottom-right            |
| Constraints         | 1 <= rows, cols <= 200                                    |
| Main Idea / Pattern | dp[j] is the cheapest path sum reaching the current cell. |
| Time Complexity     | O(mn)                                                     |
| Space Complexity    | O(n)                                                      |

### Optimal C++ Code

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        vector<int> dp(n, INT_MAX);
        dp[0] = 0;
        for (int i = 0; i < m; i++) {
            dp[0] += grid[i][0];
            for (int j = 1; j < n; j++) {
                dp[j] = min(dp[j], dp[j - 1]) + grid[i][j];
            }
        }
        return dp[n - 1];
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 9 Longest Common Subsequence

Pattern: 2D DP

LeetCode: 1143

|                     |                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Input               | text1, text2                                                                                        |
| Output              | LCS length                                                                                          |
| Constraints         | 1 <= lengths <= 1000                                                                                |
| Main Idea / Pattern | Use a table over prefixes; equal characters use the diagonal state, otherwise take max of top/left. |
| Time Complexity     | O(nm)                                                                                               |
| Space Complexity    | O(nm)                                                                                               |

### Optimal C++ Code

```
class Solution {
public:
    int longestCommonSubsequence(string a, string b) {
        int n = a.size();
        int m = b.size();
        vector<int> prev(m + 1, 0);
        for (int i = 1; i <= n; i++) {
            vector<int> cur(m + 1, 0);
            for (int j = 1; j <= m; j++) {
                if (a[i - 1] == b[j - 1]) {
                    cur[j] = prev[j - 1] + 1;
                } else {
                    cur[j] = max(prev[j], cur[j - 1]);
                }
            }
            prev = cur;
        }
        return prev[m];
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.

## PROBLEM 10 Longest Palindromic Subsequence

Pattern: 2D DP

LeetCode: 516

|                     |                                                                                          |
|---------------------|------------------------------------------------------------------------------------------|
| Input               | s                                                                                        |
| Output              | longest palindromic subsequence length                                                   |
| Constraints         | 1 <= s.size() <= 1000                                                                    |
| Main Idea / Pattern | dp[i][j] depends on matching endpoints or the better result after dropping one endpoint. |
| Time Complexity     | O(n^2)                                                                                   |
| Space Complexity    | O(n^2)                                                                                   |

### Optimal C++ Code

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n));

        for (int i = n - 1; i >= 0; i--) {
            dp[i][i] = 1;

            for (int j = i + 1; j < n; j++) {
                if (s[i] == s[j]) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[0][n - 1];
    }
};
```

**Practice:** Re-solve without looking, then change one constraint and explain which part of the pattern changes.